

UNEXT 見習生職涯沙龍

AI 開發實戰

打造第一個 AI 應用

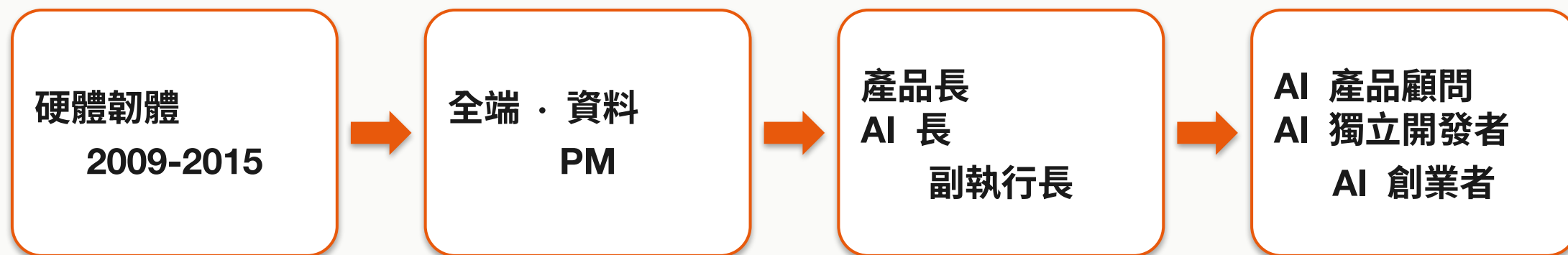
8/13 (四) 13:30-17:00 · 蔡子揚 Young



掃我 → 按 Fork

github.com/young-ai-courses/unext-ai-dev-workshop

蔡子揚 Young



接過的產業：教育 · 醫療 · 金融 · 製造

我做的事：把想法變成真的能用的產品 —— 從規格到上線

開工前：先確定這兩個帳號

① GitHub

- 程式碼的雲端硬碟
- 沒有的現在辦，用 Google 登入最快
- 之後 Codex 會直接幫你操作它

② ChatGPT + 桌面版

- Codex 就住在桌面版裡面
- chatgpt.com/download 下載
- 建議付費方案，額度比較不會中途卡住

其他都不用你辦：Vercel 用 GitHub 帳號登入、`node/git/gh/vercel` 交給 Codex 裝
唯一還要你自己跑一趟的是 Groq key → console.groq.com（免費 1,000 次／天）

Codex 桌面版：AI 直接改你電腦上的檔案

今天所有事情都在這一個對話框裡發生

- 1 chatgpt.com/download 下載 ChatGPT 桌面版（Mac / Windows）並登入
- 2 左邊選 Codex —— 它跟網頁版的差別是看得到、也改得動你本機的檔案
- 3 先按「開啟資料夾」隨便選一個空資料夾（等它幫你下載專案後再切過去）
- 4 之後每一件事都是：貼一段話 → 看它要做什麼 → 讓它做

⚠ Codex 額度依方案、模型、任務大小而變 —— 一次交辦一件小事最省
快用完時改用較小的模型，或看 usage dashboard；真的用光看 QUICKSTART 的 Plan B

Prompt 0：叫 Codex 裝環境 + 拿專案

打開 ChatGPT 桌面版 → 左邊選 Codex → 把下面這段貼進對話框 → 送出

請幫我把開發環境裝好，並把課程專案拿下來。
我不會用終端機，每一步先講你要做什麼再執行。

1. 檢查：`node -v`、`git -v`、`gh --version`、`vercel -v`
2. 缺的才裝（自己判斷 macOS 還是 Windows）：
Node.js LTS、git、gh、Vercel CLI
3. 帶我登入 gh 跟 vercel，要點同意時停下來說
4. fork 並下載這個 repo：
`github.com/young-ai-courses/unext-ai-dev-workshop`
5. 印出四個版本號 + 專案放在哪個路徑
清單以外的不要裝，需要管理員密碼先問我

它會停下來問你密碼、要你去瀏覽器點同意 —— 那是正常的，照著點
過關條件：四個版本號都印出來，而且它抓得到你的 GitHub 帳號

今天結束，你會有一個網址

- 別人打得開、能操作、真的有 AI 在裡面
- 而且你不會寫一行程式
- 不是投影片、不是截圖、不是「回家再做」
- 是一個現在就能傳給朋友的網址

我先打開我做的那個給你們看 →

今天會用到 GitHub，先講四個字

這個字	白話講就是
repo	一個專案的資料夾 —— 今天的課程資料就放在一個裡
fork	按一下，把那個資料夾複製一份，變成你的
PR	AI 改完不會直接動你的東西，先放進「等你同意」的清單
push	你改完送上去 —— 一送上去，你的網站就自動重新上線



掃我看內容

課程 repo：github.com/young-ai-courses/unext-ai-dev-workshop
裡面有：QUICKSTART（上線步驟）· 11 支 prompt · 卡住看 troubleshooting
你不必自己按 Fork —— 剛剛那支 Prompt 0 就是叫 Codex 幫你 fork 跟下載

AI 寫 code 這件事，走過三個階段

	2021 自動補全	2023 對話生成	2025 交辦執行
你做什麼	打字	問「怎麼寫」	說「我要什麼」
AI 做什麼	猜你下一行	給你一段，你貼回去	自己讀專案、改檔、開 PR
你的角色	打字的人	組裝的人	需求方 + 驗收方

Codex · Claude Code · Cursor 都在第三階段

所以瓶頸換位置了

以前的瓶頸

- 你會不會寫
- 看得懂語法嗎
- 記得住 API 嗎

現在的瓶頸

- 你到底要什麼
- 看不看得出它做錯
- 說不說得清楚

「需求講不清」比 AI 早了五十年

年代	解法	為什麼還是會出事
1970s 瀑布式	先寫超厚規格書，簽核才 開工	寫的時候還不知道要什麼
2000s 敏捷	不寫厚規格，兩週一版邊 做邊修	靠「人的常識相通」補洞
2025 SDD	規格是唯一真相，code 是它的產物	← 我們現在在這

敏捷能省規格，是因為人有常識

你跟工程師說「做登入頁」

- 他自己就知道要有密碼欄
- 要遮起來、打錯要提示
- 不能把密碼印進 log

你跟 AI 說「做登入頁」

- 它給你統計上最常見的做法
- +一堆你沒要的東西
- +漏掉你以為理所當然的

那些沒人寫在規格上的事，工程師自己補了；AI 不會 —— 它只會猜

SDD 一句話

規格是你唯一真的要親手 review 的東西
code 是它的產物

code 壞了可以重新生成一次
規格錯了，生成幾次都是錯的

所以 PRD 怎麼寫？你不用學會寫



壞消息：你不知道要寫什麼（第一次寫 PRD 的人都不知道）

好消息：你不用知道 —— LLM 知道要問什麼



這支 prompt 是今天最重要的工具

開 ChatGPT 或 Claude 的網頁版，開一個新對話貼進去（不是 Codex）

你是資深產品經理，把外行人模糊的願望，
問成工程師做得出來的需求單。

我的願望：「_____」

硬限制：不會寫程式／只有 3.5 小時／
只有一個輸入框＋一次 AI 呼叫

規則：一次只問一題，最多 6 題

只問使用者層面，不問技術問題

我說「不知道」就給我 2-3 個選項

問完產出 PRD，要有「做得完 vs 做不完」

全文（含 PRD 完整格式）在 repo → docs/02-prompts.md 的 Prompt 1，複製就好

它給你 PRD 之後：逐條 review

A 有沒有搞懂

是我要的東西嗎
有沒有自己加東西

B 夠不夠具體

「他做 X → 看到 Y」
沒有無法打勾的形容詞

C AI 做什麼

輸入、輸出格式
不確定的怎麼處理

D 做不完的呢

有沒有誠實切開
做不完的有寫下來嗎

E 有沒有踩線

個資 · 金流 · 責任
key 有沒有進 code

F 能不能打勾

三條驗收條件
當場打得勾或打得叉

最容易漏的是 D：它不會主動說「做不完」

- 你說要五個功能 → 它五個都寫進 PRD，都寫得很可行
- 因為它不知道你只有 3.5 小時、不會寫程式

所以你要主動問一句：

**「我只有 3.5 小時、不會寫程式、只有一個輸入框加一次 AI 呼叫。
重新切一次：今天做得完的一組，做不完的另外列清單」**

做不完的不要刪掉 —— demo 時你可以說「這版先做到 X，接下來長 Y」

動手：25 分鐘

- 1 想一件你每週都要重複做、很煩的事
- 2 開 ChatGPT / Claude 新對話 → 貼訪談員 prompt
- 3 它問你 6 題，你老實答
- 4 拿到 PRD → 用 A-F 六格逐條 review
- 5 不對的地方叫它改（一次只講一節）
- 6 存成你 repo 的 SPEC.md

腦袋空白的人舉手 → 我給你暖身 prompt（repo 的 Prompt 2）

接下來這條線，全自動



你只做兩個動作：把 SPEC 給 Codex、按 merge
剩下每一步都是自動發生的——這叫 CI/CD

動手：30 分鐘，讓它活著

貼在同一個地方：ChatGPT 桌面版 → Codex → 對話框 → 送出

請幫我把這個專案部署上線。我不會用終端機，
每一步先講你要做什麼，再執行。

1. 進到 starter-kit 資料夾再裝套件、再部署
一定要在 starter-kit 裡面部署，站在最外層會「成功」但打開是 404
2. 部署到 Vercel production；問到專案根目錄要選哪個，選 starter-kit，其餘用預設
3. 要加 GROQ_API_KEY 的時候停下來告訴我，我自己去 Vercel 貼
不要問我 key、不要把它寫進任何檔案或印出來
4. 我說加好了，你再部署一次 production 讓它生效
5. 把網址給我，並確認打開不是 404

key 你自己到後台貼，不經過 AI 對話 —— 這是今天唯一不能外包的事

網站 → Vercel 專案 Settings → Environment Variables | 排程 → GitHub repo Settings →
Secrets and variables → Actions

把 SPEC 交給 Codex 改

規格是你寫的，動手的是它 —— 你只負責看它做對沒有

- 1 在 Codex 貼上你 review 過的 SPEC，加一句「先複述你的理解，等我說開始」
- 2 看它複述得對不對 —— 不對就先講清楚，別讓它開工
- 3 它改完會給你 diff（哪幾個檔、改了什麼）→ 你看過再接受
- 4 接受後 push → Vercel 自動重新部署，30 秒後重新整理你的網址

你做的東西，長這樣

橘色是你的（改得動） · 灰色是租來的服務（只能呼叫）



①②③ 是你的，你改得動；④⑤ 是租來的，你只能照它的規則呼叫

想法落在哪一層，就叫 AI 動那一層

你不必知道要改哪個檔 —— 講你想看到什麼就好

你的想法	落在	你就這樣跟 Codex 講
畫面上的字、顏色、按鈕	① 頁面	「送出按鈕的字改成『幫我整理』」
AI 回答的內容或格式	③ 伺服器	「回答多一欄負責人，沒寫就填未指定」
要記住上次打了什麼	⑤ 資料庫	「我要記住上次打的，該加什麼？」

說得出「從什麼變成什麼」→ 直接講，這叫具體

不知道該動哪裡 → 先問路：「我要 __，該加什麼？」，這叫抽象

那個 AI 在你的網站哪裡？



為什麼 key 不能放前端：任何人打開瀏覽器的原始碼就看得得到它
所以前端只跟「你自己的伺服器」說話，由伺服器拿著 key 去問 AI

換一個 AI = 改一段話（不用改程式邏輯）

跟 Codex 說：把 AI 的角色設定換成這段 —

你是一位嚴謹的會議秘書。

把使用者貼進來的筆記整理成三類：

待辦事項 / 已做的決議 / 需要再確認的事。

用表格輸出，欄位是「類別 | 內容 | 負責人」。

沒寫負責人的填「未指定」，不要自己猜是誰。

不確定的放進「需要再確認」，不要編。

「不要自己猜」「不要編」這兩句很重要 —— 不寫它就會編

改東西的兩條紀律

① 一次只講一件事

- ✗ 「幫我做一個會議整理器」
- ✗ 「介面弄漂亮一點」
- ✓ 「把標題改成 X，提示改成 Y」

② 壞了先退回，不要接著補

- ✗ 「幫我修好」 → 它會一直加東西，越補越難救
- ✓ 「先找出上一個能動的版本，說明要退哪些檔，等我確認」
- 退回去之後，把需求重講一次

退回去不算失敗 —— 版本控制的整個意義，就是讓你敢改

「那我打的東西存哪？」

- 今天沒有資料庫 —— 這是刻意的
- 「輸入 → AI → 看結果」這形狀已經有用，今天做得完
- 加資料庫要裝服務、接環境變數，至少 40 分鐘

要存東西的時候，用 Supabase

你不用先學它 —— 跟 Codex 說：「我要記住上次打的，幫我接 Supabase」

⚠ 唯一要記住的一件事：建完表一定要開 RLS，否則任何人都讀得走你的資料

你剛才做的事，有名字

我用來教學的五層地圖（不是官方標準名詞）—— 你今天走過前三層

- | | | | |
|---|----------------------------|--|----------------------------|
| 1 | Prompt Engineering | | 你寫的那些 prompt |
| 2 | Context Engineering | | 你 review 過的 SPEC.md ← 就是這層 |
| 3 | Harness Engineering | | 那個綠勾勾 CI ← 也是這層 |
| 4 | Loop Engineering | | 讓系統自己一輪一輪推進（還沒做） |
| 5 | Graph Engineering | | 多個 AI 分工協作（還沒做） |

123 今天你親手做過了

上台 2 分鐘，講這三件事

- 1 我原本要花多久做這件事
- 2 這個東西幫我做掉了哪一段
- 3 今天沒做完的部分，接下來會怎麼長

github.com/young-ai-courses/unext-ai-dev-workshop

QUICKSTART 上線步驟從頭到尾

docs/02-prompts 今天用到的每一支 prompt

docs/03-troubleshooting 卡住了看這裡